Module 7 – Advanced GitOps with ArgoCD on Amazon EKS Anywhere

Production Deployment Strategies with ArgoCD

Managing Hundreds of Kubernetes Applications Safely across enterprise-scale environments on VMware vSphere.

1. Helm GitOps
2. Kustomize
3. Secrets
4. Drift Detection
5. Progressive Delivery

Learning Objectives

Why Basic GitOps Isn't Enough

At enterprise scale, simple GitOps pipelines break down. Real production environments demand far more sophistication — across hundreds of clusters, thousands of applications, and multiple teams with strict compliance requirements.

100+ Clusters

Managing fleet-wide deployments across diverse environments

1000+ Applications

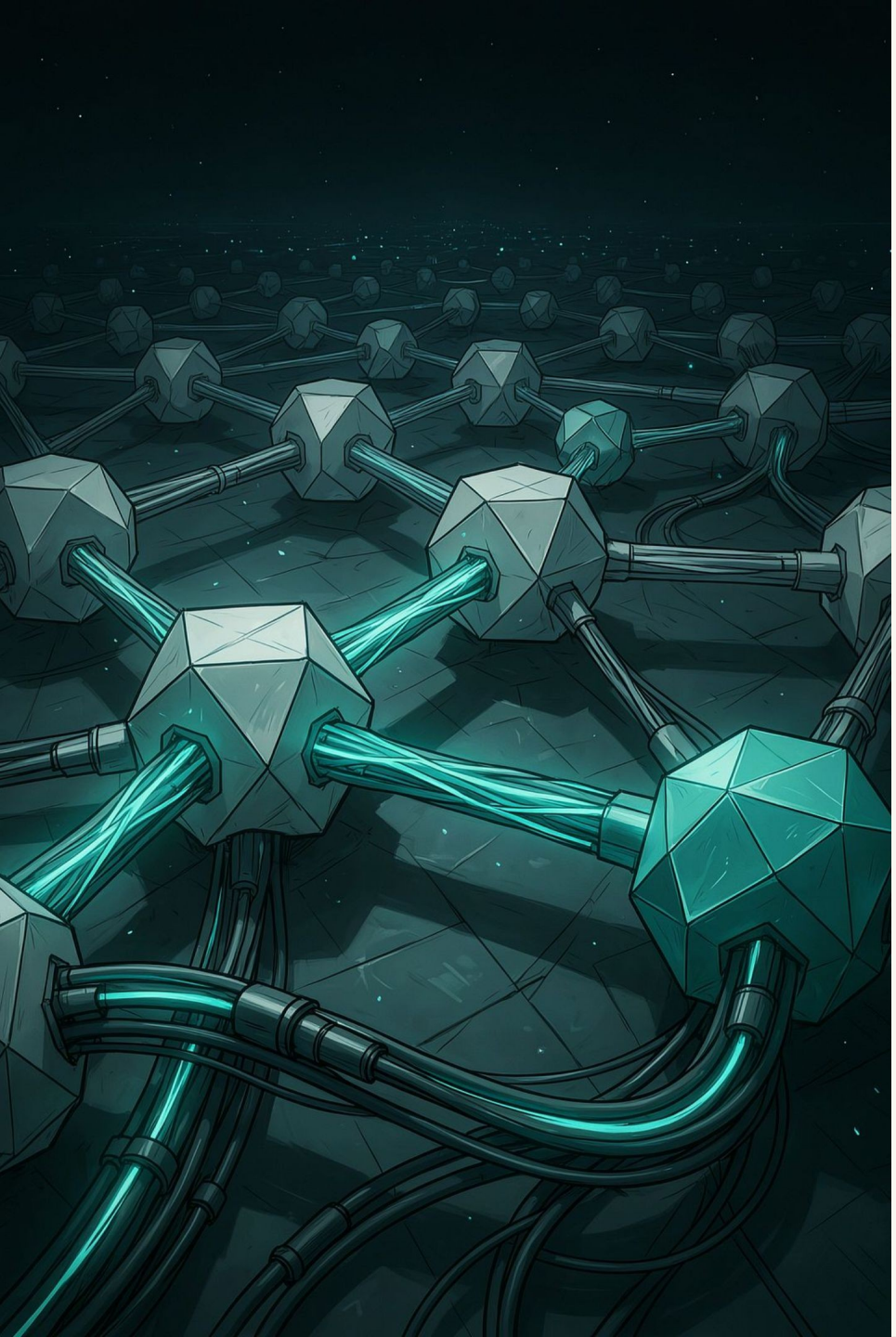
Different customers, teams, and release cadences

Security & Compliance

Zero downtime, audit trails, and strict access controls

Multi-Team

Coordinating across platform, app, and infra teams simultaneously



Enterprise GitOps Architecture

ArgoCD acts as the central control plane — continuously reconciling desired state from Git against live cluster state, enabling automatic rollback and a full audit trail.

Desired State

Git is the single source of truth

Continuous Reconciliation

ArgoCD constantly syncs live state

Automatic Rollback

Drift triggers immediate correction

Audit Trail

Every change is tracked in Git history

GitOps Repository Design

Repository Structure

```
gitops/
  apps/
    backend/
    frontend/
    payments/
  platform/
    ingress/
    monitoring/
    cert-manager/
  clusters/
    dev/
    test/
    prod/
  shared/
    helm-charts/
    kustomize/
```

Design Principles

→ Separation of Concerns

Infrastructure and application configs live independently

→ Environment-Specific Config

Dev, test, and prod each have isolated configuration paths

→ Reusable Components

Shared Helm charts and Kustomize bases reduce duplication

Helm GitOps — How It Works

ArgoCD's Helm integration differs fundamentally from the Helm CLI. ArgoCD renders Helm templates server-side and manages the resulting manifests directly — giving you GitOps-native lifecycle management without `helm install` or `helm upgrade` commands.

Helm CLI

- Imperative — you run commands manually
- State stored in cluster Secrets
- No automatic drift correction

ArgoCD Helm

- Declarative — Git drives all changes
- State tracked in Git history
- Continuous reconciliation & rollback

Helm Value Hierarchy

ArgoCD applies Helm values in a layered inheritance model. Base defaults are progressively overridden by environment, application, and cluster-specific values — enabling maximum reuse with precise control.

1

values.yaml

Global defaults — shared across all environments

2

values-dev / test / prod.yaml

Environment overrides — replicas, resource limits, feature flags

3

Application Overrides

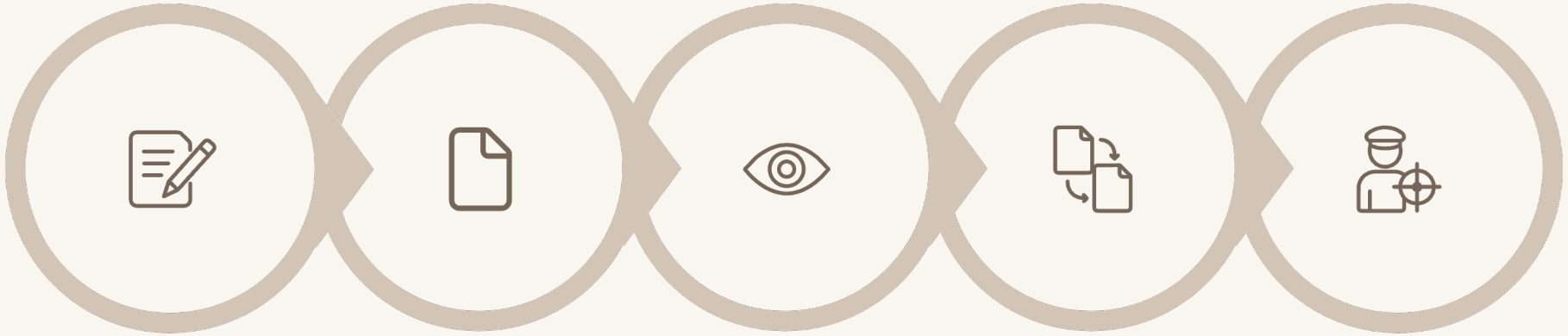
Per-app customizations applied on top of environment values

4

Cluster Overrides

Final layer — cluster-specific endpoints, secrets, and topology

Helm GitOps Workflow



Update Values

Git Push

ArgoCD Detects

Sync Triggered

Helm Render

Auto Sync

ArgoCD automatically applies changes on every Git commit

Manual Sync

Require explicit approval before applying to production

Rollback

Instantly revert to any previous revision stored in Git

Revision History

Full audit trail of every deployment and change

Helm Best Practices

Small, Reusable Charts

Avoid monolithic charts. Compose small, versioned charts with semantic versioning and OCI registry distribution.

Signed Charts

Use chart signing and verification to ensure supply chain integrity in production environments.

Chart Dependencies

Declare dependencies explicitly. Pin versions to prevent unexpected upstream changes.

Production Tips

> Avoid giant `values.yaml` — split by concern

> Separate secrets from values entirely

> Always use immutable image tags in production

Kustomize Fundamentals

Kustomize takes a different philosophy from Helm — no templating language, no new syntax. It works natively with plain Kubernetes YAML using an overlay and patch model, making it ideal for teams who want to stay close to raw Kubernetes manifests.

No Templates

Pure YAML — no Go templates or custom syntax to learn

Native Kubernetes

Built into `kubectl` — no extra tooling required

Overlay Based

Layer environment-specific changes on top of a shared base

Patch Based

Apply surgical changes without duplicating entire manifests

Kustomize Directory Structure

Directory Layout

```
base/
  deployment.yaml
  service.yaml
  kustomization.yaml
overlays/
  dev/
    kustomization.yaml
  test/
    kustomization.yaml
  prod/
    kustomization.yaml
```

How It Works

1 Base Layer

Reusable, environment-agnostic Kubernetes manifests shared across all targets

2 Overlay Layer

Environment-specific patches applied on top — dev, test, and prod each customize independently

3 Final Manifest

ArgoCD renders the merged output and applies it to the target cluster

Kustomize Features

Kustomize provides a rich set of built-in transformers and generators that cover the most common Kubernetes customization needs — without a single line of template logic.

Strategic Merge & JSON Patch

Apply targeted changes to any field in a manifest without duplicating the entire resource

Name Prefix & Namespace Injection

Automatically scope resources per environment with prefixes, suffixes, and namespace overrides

Image Replacement

Override image tags per environment — pin prod to immutable digests, use `latest` in dev

ConfigMap & Secret Generator

Auto-generate ConfigMaps and Secrets from files or literals, with automatic hash-based rolling updates

Helm vs. Kustomize

Choosing the right tool depends on your team's needs. Here's how they compare across the dimensions that matter most in enterprise GitOps.

Dimension	Helm	Kustomize
Learning Curve	Moderate — Go templates required	Low — pure YAML
Templating	Full Go template engine	None — patch-based only
Reuse	High — chart libraries, OCI	Moderate — base overlays
Flexibility	Very high — full logic support	High — surgical patches
Secrets	External plugins needed	Secret Generator built-in
ArgoCD Support	Native, first-class	Native, first-class
Enterprise Usage	Very common for complex apps	Common for platform configs

Decision Matrix — Helm, Kustomize, or Both?

Choose Helm When...

- You need complex conditional logic or loops in manifests
- Distributing reusable charts across many teams or customers
- Managing third-party app deployments from public chart repos

Choose Kustomize When...

- You want to stay close to raw Kubernetes YAML
- Managing platform configs with simple environment differences
- Teams prefer no templating language overhead

Combine Both When...

- Third-party Helm charts need environment-specific patches
- You want Helm's packaging with Kustomize's overlay precision
- ArgoCD supports this hybrid natively — no extra tooling needed

Key Takeaways

01

Scale Requires Structure

Enterprise GitOps demands deliberate repo design, separation of concerns, and layered configuration strategies

02

ArgoCD Transforms Helm

ArgoCD's Helm integration is declarative and GitOps-native — fundamentally different from imperative Helm CLI usage

03

Kustomize Stays Native

No templates, no new syntax — Kustomize's overlay model keeps manifests close to raw Kubernetes YAML

04

Hybrid Is Valid

Helm and Kustomize are not mutually exclusive — ArgoCD supports combining both for maximum flexibility

Next: Module 8 — Secrets Management with External Secrets Operator and AWS Secrets Manager on EKS Anywhere

Module 7 — Reference & Resources

Official Documentation

ArgoCD Docs

argo-cd.readthedocs.io — Helm & Kustomize integration guides

EKS Anywhere

anywhere.eks.amazonaws.com — VMware vSphere cluster setup

Kustomize.io

Official reference for overlays, patches, and generators

Topics Covered in This Module

- Helm GitOps
- Value Hierarchy
- Helm Workflow
- Helm Best Practices
- Kustomize Fundamentals
- Kustomize Features
- Helm vs Kustomize
- Decision Matrix